



Client-side Technologies

Dr. Niveen Nasr El-Den
iTi



Day 8



JavaScript Fundamentals cont.

JavaScript Cookies



Cookies

- Cookies are **small text** strings that you can store on the computers of people that visit your Web site.
- Cookies were originally invented by Netscape to give '**memory**' to web servers and browsers.
- Normally, cookies are **simple variables** set by the server in the browser and returned to the server every time the browser accesses a page on the same server.
- A cookie is not a script, it is a mechanism of the **HTTP** server accessible by both the client and the server.

Need Of Cookies

- HTTP is a **state-less** protocol; which means that once the server has sent a page to a browser requesting it, it doesn't remember any thing about it.
- The HTTP protocol, is responsible for arranging:
 - Browser requests for pages to servers.
 - The transfer of web pages to your browser.

Need Of Cookies

- *Stateless protocols* have the **advantage** that they require fewer resources on the server
-- the resources are pushed into the client.
- But the **disadvantage** is that the client needs to tell the server enough information on each request to be able to get the proper answer.
- As soon as personalization was invented, this became a major problem.
- **Cookies** were invented to solve this problem.



HTTP cookie

=

Web cookie

=

Browser cookie

Cookies

- ***Cookies*** are a method for a server to ask the client to store arbitrary data for use in future connections.
- They are typically used to carry persistent information from page to page through a user session or to remember data between user sessions.
- With JavaScript, you can create and read cookies in the client-side without resorting to any server-side programming.
- A cookie may be written and accessed by a script but the cookies themselves are simply passive **text strings**.

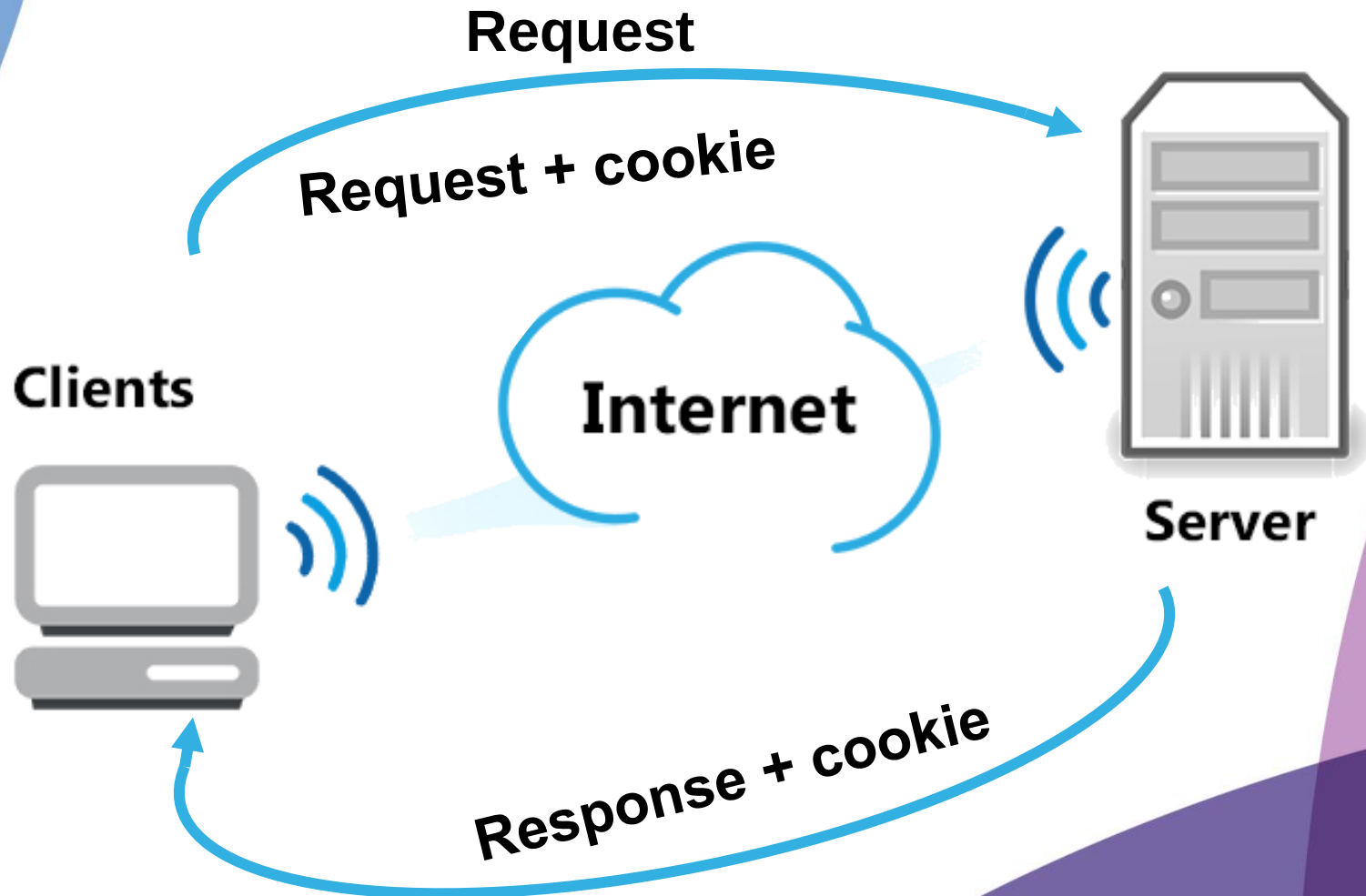
Types Of Cookies

- Cookies has two types:
 - **Session Cookies/ Non-persistent** : These cookies reside on the Web browser and have *no expiry date*. They expire as soon as the visitor closes the Web browser.
 - **Persistent Cookies**: These cookies have an *expiry date*, are stored on a visitor's hard drive and are read by the visitor's browser each time the visitor visits the Web site that sent the cookie

Benefits of Cookies

Session Management

- **Authentication**
 - no longer need to enter password
 - Greeting people by name.
 - **Saving time for returning visitors**
 - The user does not have to re-enter information
- **Research websites.**
- **Maintaining state**
 - Adventure games that use cookies to keep track of pertinent character data and the current state of the game.
- **Shopping carts**
 - By storing data as you move from one page (or frame) to another.
- User preferences, themes, and other settings → **Personalization**
- Tracking Recording and analyzing user behavior



Cookies Limitations

- All Browsers are preprogrammed to allow a total of **300** Cookies, after which automatic deletion based on expiry date and usage.
- Each individual domain name (site) can store **20** cookies.
- Each cookie having a maximum size of **4KB**.

Cookies Facts

- A server can set, or deposit, a cookie only if a user visits that particular site.
 - i.e. one domain cannot deposit a cookie for another, and cross-domain posting is not possible.
- A server can retrieve only those cookies it has deposited.
 - i.e. one server cannot retrieve a cookie set by another.
- Cookies can be retrieved only by the Web site that created them. Therefore any cookie you create is safe from view of other Web sites.
- Cookies are sent with every request, so they can worsen performance (especially for mobile data connections).

Cookies Securing Facts

- Highly unreliable, from a programming perspective.
 - It's like having your data stored on a hard drive that sometimes will be missing, corrupt, or missing the data you expected.
- Cookie security is such that only the originating domain can ever use the contents of your cookie "*Same-origin policy*".
- Cookies just identify the computer being used, not the individual using the computer.
- Cookie files stored on the client computer are easily read by any word processing program, text editor or web browsing software unless an encryption mechanism is applied.

Cookies False Claims

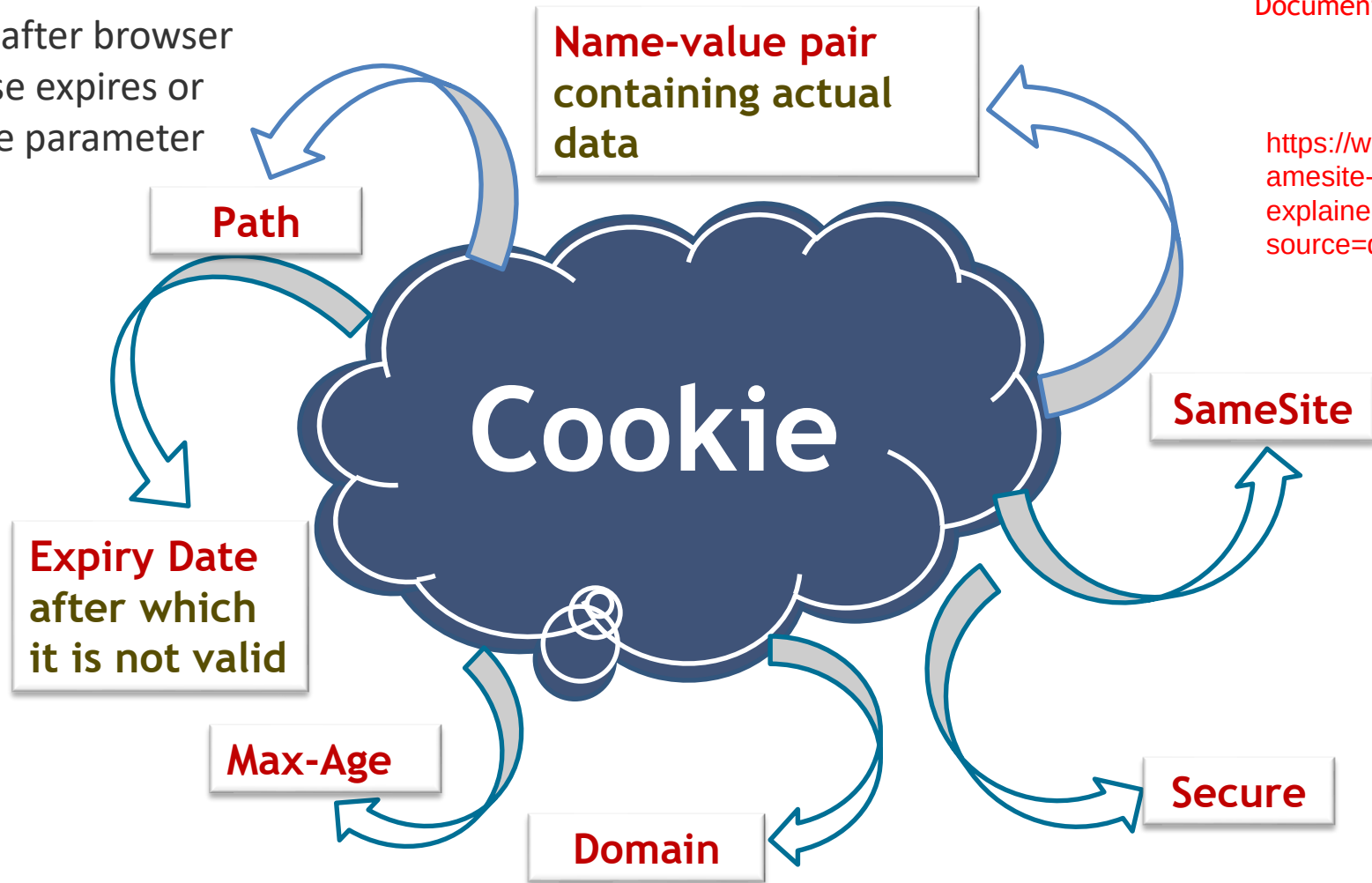
- Cookies are like **worms** and **viruses** in that they can erase data from the user's hard disks
- Cookies generate **popups**
- Cookies are used for **spamming**
- Cookies are only used for **advertising**

Cookies Parameters

<https://developer.mozilla.org/en-US/docs/Web/API/Document/cookie>

https://web.dev/samesite-cookies-explained/?utm_source=devtools

To let cookies survive after browser close use expires or max-age parameter



Cookies Parameters

<https://developer.mozilla.org/en-US/docs/Web/API/Document/cookie>

Parameter	Description	Example
name=value	This sets both cookies name and its value. The cookie value string can use <code>encodeURIComponent()</code> to ensure that the string is in a valid format and does not contain any disallowed char in cookie values e.g. commas, semicolons, or whitespace.	username=JavaScript
expires=date	This optional value set the date that the cookie will expire on. The date should be in the GMT format. If the expires value is not given, the cookie will be destroyed the moment the browser is closed	expires= today.toUTCString()
max-age=sec	Similar to expires but is a number of seconds till the cookie disappears. It has priority over expires If neither expires nor max-age specified it will expire at the end of session.	max-age=60*60*60*5 // 5 hours

Working with Cookies

- Cookies can be *created*, *read* and *deleted* by JavaScript, under these conditions:
 1. The user's navigator must be cookie-enabled. This can be checked using "*navigator.cookieEnabled*" property .
 2. The cookie(s) that you set or accept are only accessible at pages with a *matching domain name, matching path*.
 3. The cookies must not have reached or passed their expiry date.
- When these criteria are met the cookies become available to JavaScript via the *document.cookie* property.

Creating a Cookie

- Assigning a value to the *document.cookie* property

document.cookie="name=value";

document.cookie="name=value;expires=date";

```
<head>
  <script language="JavaScript">
    document.cookie = "myCookie =" +
      encodeURIComponent("This is my Cookie");
    window.alert("myCookie=" +
      encodeURIComponent("This is my Cookie"));
  </script>
</head>
```

Creating a Cookie

- Assigning a value to the *document.cookie* property

document.cookie="name=value;expires=date"

```
<head>
  <script language="JavaScript">
    var myDate = new Date();
    document.cookie = "myCookie=" +
                      encodeURIComponent("This is my Cookie") +
                      ";expires=" + myDate.toGMTString();
  </script>
</head>
```

Displaying a Cookie

- **Retrieve created Cookie value**
 - Extract the name and value of the cookie to two variables.
 - The document.cookie will keep a list of name=value pairs separated by semicolons, where name is the name of a cookie and value is its string value
 - We use strings' *split()* function to break the string into key and values.

```
<head>
  <script language="JavaScript">
    var newCookie = document.cookie;
    var cookieParts = newCookie.split("=");
    var cookieName = cookieParts[0];
    var cookieValue = decodeURIComponent(cookieParts[1]);
    window.alert(cookieName);
    window.alert(cookieValue);
  </script>
</head>
```

Clearing a Cookie

- If the user logs out or explicitly asks not to save his or her username in a cookie, hence, you need to delete a cookie to remove a username cookie.
- Simply reassign the cookie, but set the expiration date to a time has already passed.

```
<head>  
  <script language="JavaScript">  
    var newDate = new Date();  
    newDate.setTime(newDate.getTime() - 86400000);  
    document.cookie = "myCookie=;expires="+ newDate.toUTCString();  
  </script>  
</head>
```

Multiple Cookies

- Most Web browsers set limits on the number of cookies or the total number of bytes that can be consumed by the cookies from one site.
- **Creating Multiple cookies**
 - Assign each cookie in turn to the `document.cookie` object and ensure that each cookie has a different name, and may have a different expiration date and time.
- **Accessing Multiple Cookies**
 - more complicated since accessing `document.cookie`, there will be a series of cookies separated by semicolons;

```
CookieName=firstCookieValue;secondCookieName=secondCookieValue;etc.
```



When we **update**
or **delete** a cookie,
we should use exactly
the same **path** and
domain options as
when we set it.

Creating a Cookie Function Library

- Working with cookies requires a lot of string and date manipulation, especially when accessing existing cookies when multiple cookies have been set.
- To address this, you should create a small cookie function library for yourself that can:
 - create
 - access
 - delete

cookies

without needing to rewrite the code to do this every time.

Creating a Cookie Function Library

- **getCookie (cookieName)**
Retrieves a cookie value based on a cookie name.
- **setCookie (cookieName,cookieValue[,expiryDate])**
Sets a cookie based on a cookie name, cookie value, and expiration date.
- **deleteCookie (cookieName):**
Deletes a cookie based on a cookie name.
- **allCookieList ():**
returns a list of all stored cookies
- **hasCookie (cookieName)**
Check whether a cookie exists or not

Cookies

were once used for general
client-side storage.

Now it is recommended to use
Modern Storage APIs.

Web Storage API

&

IndexedDB.

Cookie

is a small piece of data that a server sends to the user's web browser.

The browser may store it and send it back with later requests to the same server.

Cookie

is used to tell if two requests came from the same browser keeping a user logged-in etc..



Error Object

(Built-in Object)

Error Object Creation

- Whenever an error occurs, an instance of **error** object is created to describe the error.
- Error objects are created either by the environment (the browser) or by your code.
- Developer can create Error objects by 2 ways:
 - **Explicitly:**
 - `var newErrorObj = new Error();`
 - thrown using the throw statement
 - **Implicitly:**
 - Occurs automatically during runtime;
 - Generated by JS engine when rules are violated

Error Object Construction

- Error constructor
 - `var e = new Error();`
- More than Six additional Error constructor ones exist and they all inherit Error:

EvalError	Raised by eval when used incorrectly
RangeError	Numeric value exceeds its range
ReferenceError	Invalid reference is used
SyntaxError	Used with invalid syntax
TypeError	Raised when variable is not the type expected
URIError	Raised when <code>encodeURIComponent()</code> or <code>decodeURIComponent()</code> are used incorrectly

- Using *instanceOf* when catching the error lets you know if the error is one of these built-in types.

Error Object Properties

Property	Description
description	Plain-language description of error (IE only)
fileName	URI of the file containing the script throwing the error
lineNumber	Source code line number of error
message	Plain-language description of error (ECMA)
name	Error type (ECMA)
number	Microsoft proprietary error number

Error Object Standard Properties

- **name** → The name of the error constructor used to create the object
 - Example:
 - `var e = new EvalError('Oops');`
 - `e.name;`
- "EvalError"
- **Message** → Additional error information:
 - Example:
 - `var e = new Error('jaavcsritp is _not_ how you spell it');`
 - `e.message`
- " jaavcsritp is _not_ how you spell it"

Example!



Error Handling

JavaScript Error Handling

- There are two ways of catching errors in a Web page:

1.try...catch statement.

2.onerror event.

try...catch Statement

- The try...catch statement allows you to test a block of code for errors.
- The **try** *block* contains the code to be run.
- The **catch** *block* contains the code to be executed if an error occurs.
- Syntax

```
try {  
    //Run some code here  
}  
catch(err) {  
    //Handle errors here  
}
```

Implicitly an Error object
“err” is created

If an exception happens in “scheduled” code, like in setTimeout or any asynchronuous behavior, then try..catch won’t catch it

try...catch Statement

try {

✓ no error.

✓ no error.

an error! *control is passed to the catch block here.*

this will never execute.

}

catch(exception)

{

✓ error handling code is run here

}

✓ execution continues from here.

Example!

throw Statement

- The throw statement allows you to create an exception.
- Using throw statement with the try...catch, you can control program flow and generate accurate error messages.
- **Syntax**
`throw(exception)`
- The exception can be a **string**, **integer**, **Boolean** or an **object**

try...catch & throw Example

```
try{  
    if(x<100)  
        throw "less100"  
    else if(x>200)  
        throw "more200"  
}  
catch(er){  
    if(er=="less100")  
        alert("Error! The value is too low")  
    if(er == "more200")  
        alert("Error! The value is too high")  
}
```

Example!

Adding the *finally* statement

- If you have any functionality that needs to be processed regardless of success or failure, you can include this in the *finally* block.

try...catch...finally Statement

try {

- ✓ no error.
- ✓ no error.
- ✓ no error.

catch(exception)

{

- ✓ ~~error handling code will not run.~~

}

finally {

- ✓ This code will run even there is no failure occurrence.

}

- ✓ execution will be continued.

try...catch...finally Statement

try {

- ✓ no error.
- ✓ no error.

an error! *control is passed to the catch block here.*
this will never execute.

}

catch(exception)

{

- ✓ error handling code is run here
- ✓ error handling code is run here
- ✓ error handling code is run here

}

finally {

- ✓ This code will run even there is failure occurrence.

}

- ✓ execution will be continued.

Example!

try...catch...finally Statement

try {

- ✓ no error.
- ✓ no error.

an error! *control is passed to the catch block here.*
this will never execute.

}

catch(exception)

{

- ✓ error handling code is run here

an error!

~~error handling code is run here will never execute.~~

}

finally {

- ✓ This code will run even there is failure occurrence.

}

~~execution wont be continued.~~

Example!

onerror Event

- The old standard solution to catch errors in a web page.
- The *onerror* event is fired whenever there is a script error in the page.
- onerror event can be used to:
 - Suppress error.
 - Retrieve additional information about the error.

Suppress error

```
function supError() {  
    alert("Error occured")  
}  
window.onerror=supError
```

OR

```
function supError() {  
    return true; //or false;  
}  
  
window.onerror=supError
```

The value returned determines whether the browser displays a standard error message.

true the browser does **not** display the standard error message.

false the browser **displays** the standard error message in the JavaScript console

Retrieve additional information about the error

onerror=handleErr

```
function handleErr(msg,url,l,col,err) {  
    //Handle the error here  
    return true; //or false;  
}
```

where

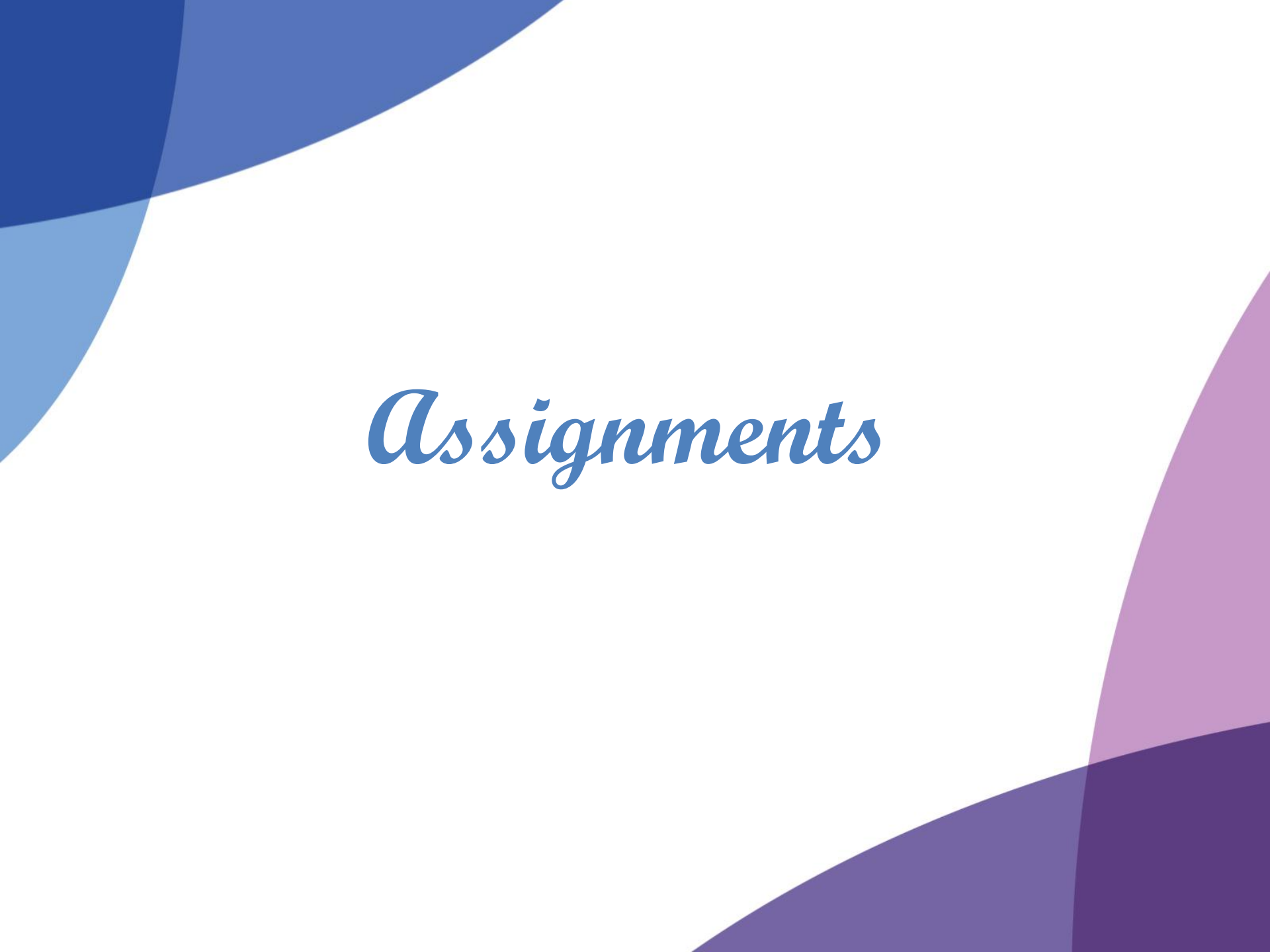
msg → Contains the message explaining why the error occurred.

url → Contains the url of the page with the error script

l → Contains the line number where the error occurred

col → Column number for the line where the error occurred

err → Contains the error object



Assignments